UNIVERSITY
OF COLOGNE

Introduction to NLP and (L)LMs

24.04.2026, 12:00-16:00

Hanna Wołoszyn (hwoloszy@uni-koeln.de)

Self Learning Systems Lab @ Uni Cologne

“Inside the Black Box: A Deep Dive into Large Language Models", Summer Semester 2026, University of Cologne

Time plan

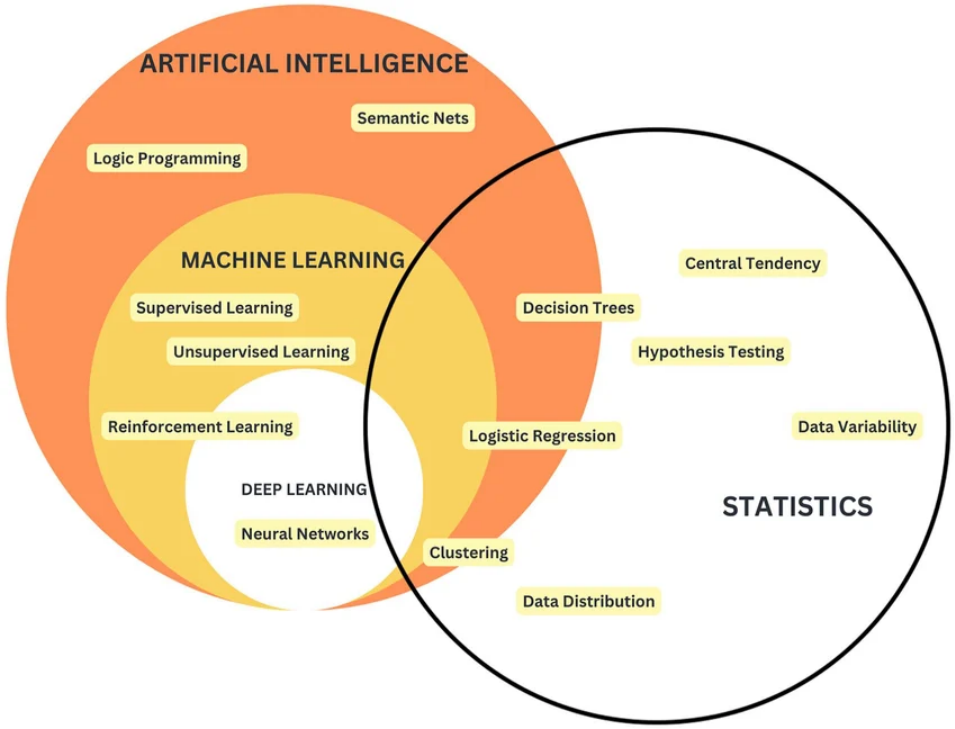
1. ~~Introduction (17.04.2026, 12:00-14:00)~~
2. **Introduction to NLP and (L)LMs (24.04.2026, 12:00-16:00)**
3. How Large Language Models Represent Meaning (08.05, 12:00-16:00) (ROOM CHANGE)
4. Presentations + The Hidden Labor, Research Crisis & Ethical Costs (22.05.2026, 12:00-16:00)
5. Presentations + Transparency & Open Source (12.06, 12:00-16:00)
6. Presentations + Hands on Research (19.06, 12:00-15:00)

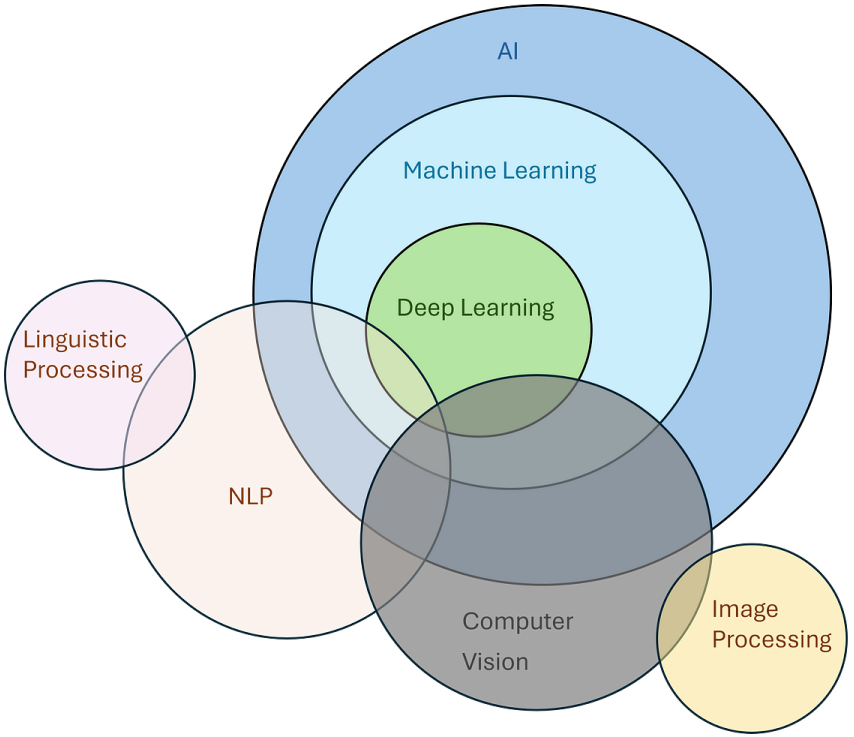
Agenda

1. Presentation of the paper “Attention Is All You Need” by Deepak Sorout and Dayoon Choi chaired by Wayne Lee
2. Recap From the Last Session
3. **Part I: NLP**
    1. Issues, Goals, and Applications of NLP
    2. Text Mining, Preprocessing and Language Modelling
4. **Part II: (Large) Language Models**
    1. Types, Training, and Applications

Recap: AI Buzzwords

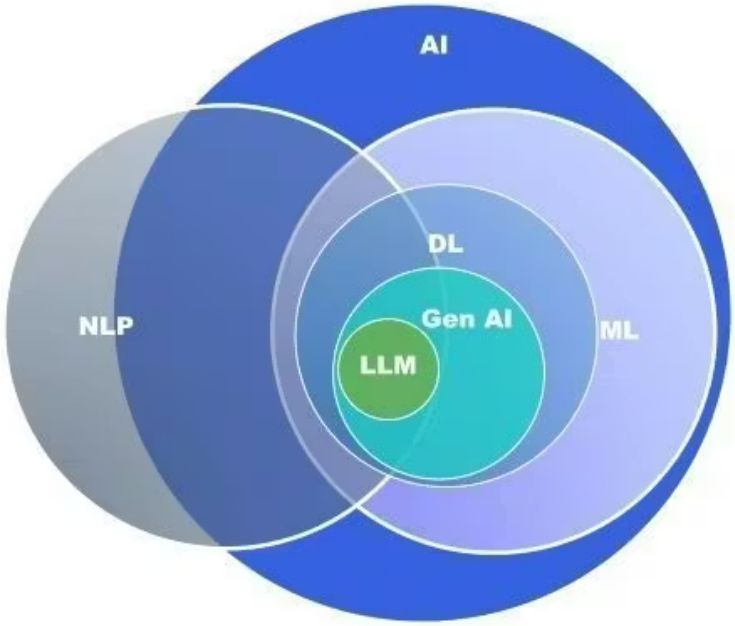
- LM, LLM, AI, ML, DL, NLP, CL, NN, GPT, NWP, CNN, RNN … wtf?





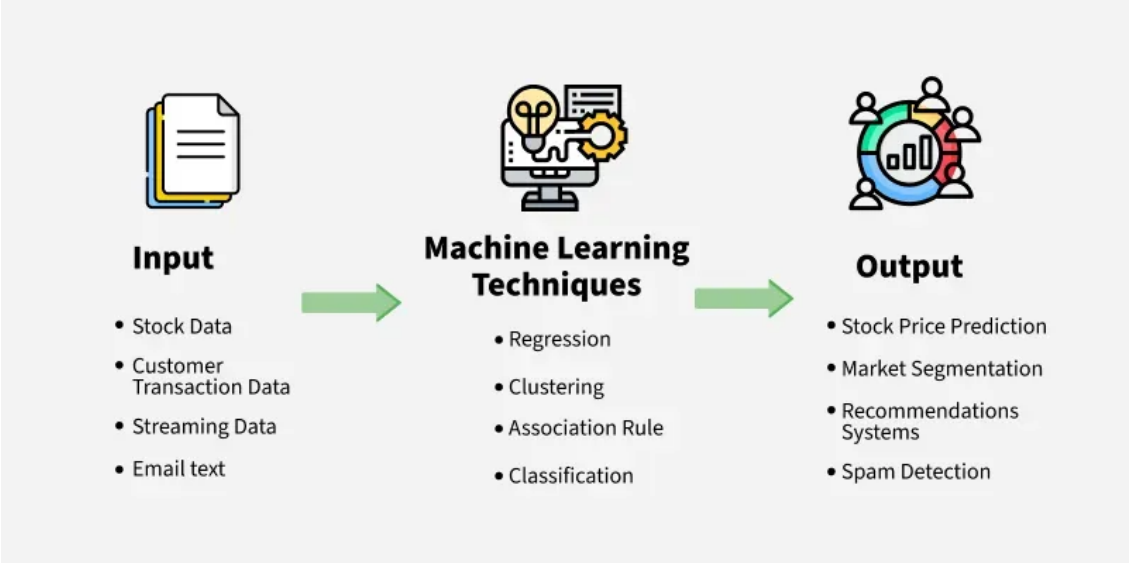
Recap: AI Buzzwords

AI-LLM Spectrum



Recap: Artificial Intelligence and Machine Learning

- AI: branch of computer science, that deals with **intelligent agents**, systems that can **reason**, **learn**, and **act autonomously**
  - LLMs and chatbots are only one of many applications of AI
- ML: subset of AI, algorithms that can make **predictions based on the data**



Recap: Deep Learning and Neural Networks

- DL: type of ML that uses Neural Networks
  - NN can process more complex patterns than traditional ML algorithms

House prices from simple inputs like **size**, **location**, and **number of rooms**:

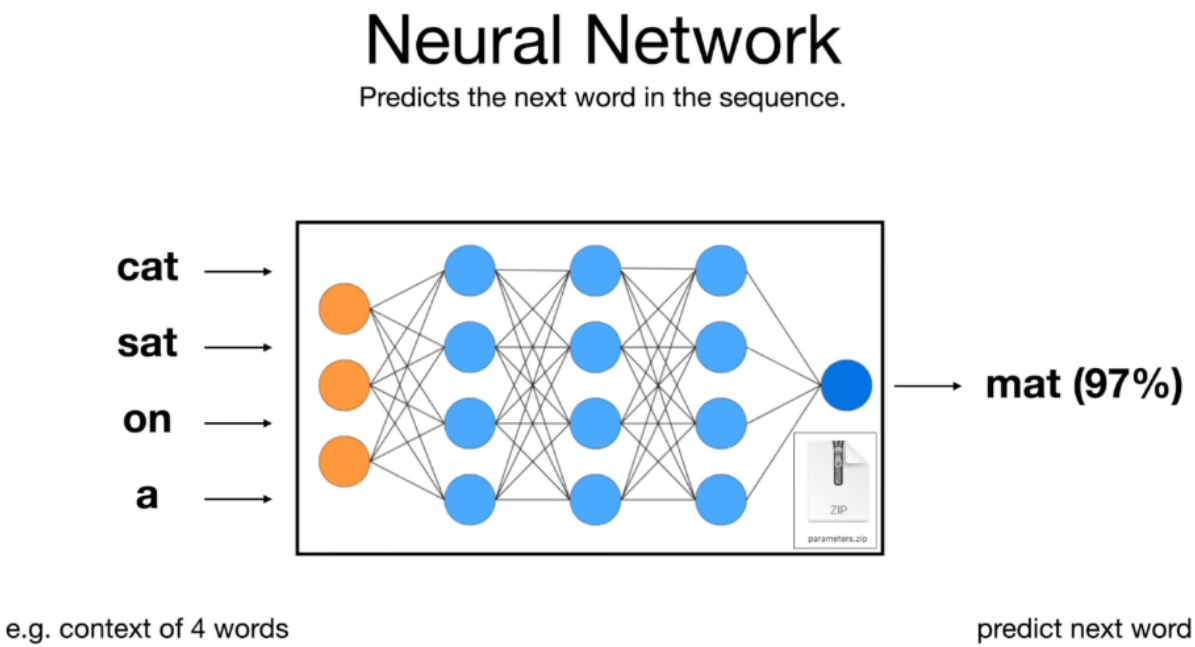
linear regression

Classifying images of cats and dogs from inputs like **texture**, **shape**, **edges** (complex visual patterns):

NNs

Recap: Neural Network

- A ML algorithm inspired by the human brain
- Layers of "neurons" that process information
- Learn from data by adjusting "weights" between nodes (learning from errors)
- Pattern recognition, predictions, data classification for computer vision, natural language processing, and speech recognition



Recap: Language Models, Natural Language Processing, Computational Linguistics

- AI, ML, DL, and NNs: how machines **learn**
- NLP, and LMs: how machines **work with human language**
- CL?: formal/computational description of languages as a system

Computational Linguistics → NLP → Language Models

language theory → language processing → language generation

Recap: Linguistics vs Computational Linguistics

- Lingusitics: the rules followed by languages as a **system**, explains the language rules

The **red** **car**

- CL: **formal** or **computational** description of rules that languages follow, formalizes those rules for computers

Adjectives come before the **nouns**

Recap: Generative Pretrained Transformer

- GPTs are based on a **deep learning architecture** called the **transformer**

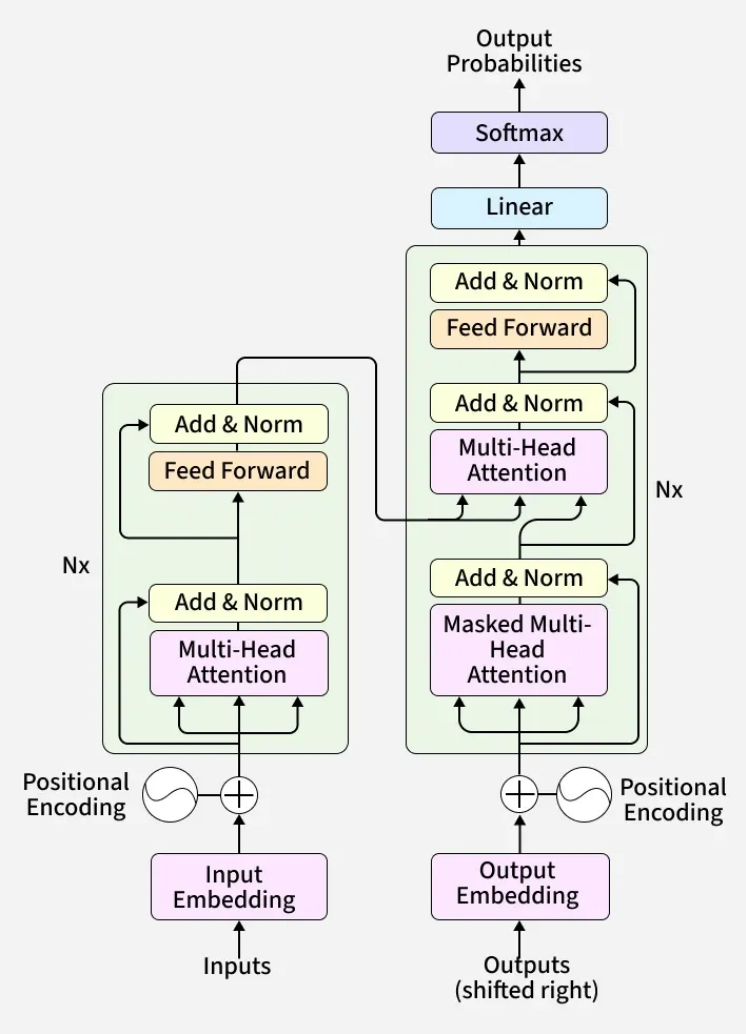
Generative → a part of generative AI

Pre-trained → it learned from a lot of existing text before being used

Transformer → the neural network architecture it is based on

Recap: Transformer Architecture

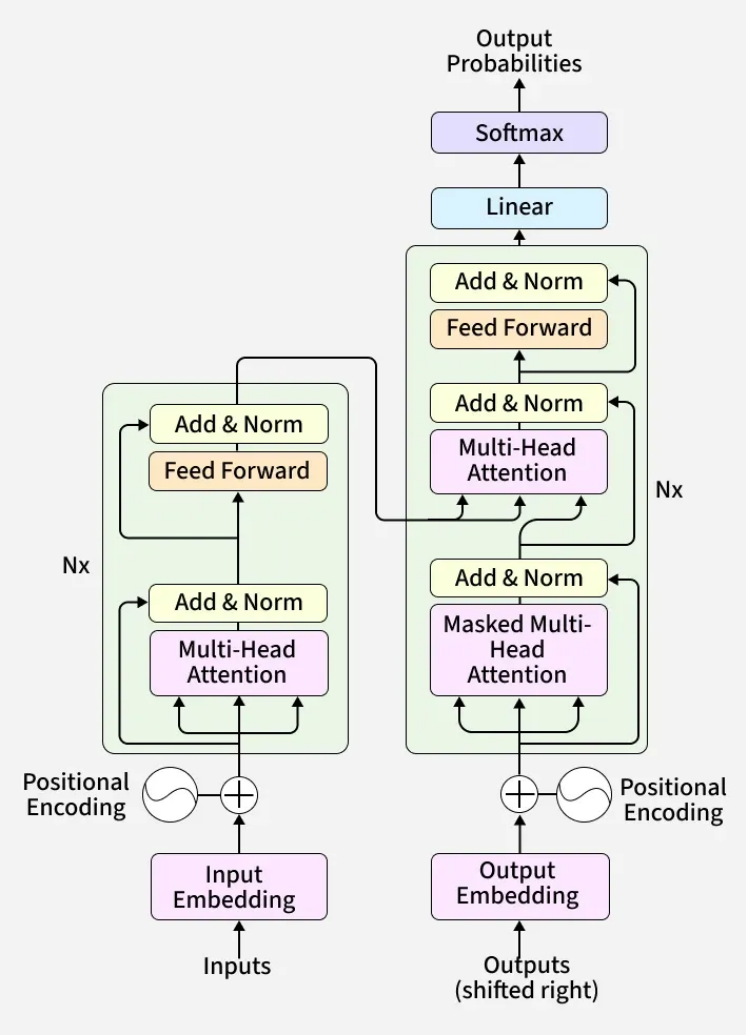
Transformer is a **deep learning neural network** architecture based on the **multi-head attention mechanism**, in which text is converted to numerical representations called tokens, and each token is converted into a vector via lookup from a word embedding table. At each layer, each token is then contextualized within the scope of the context window with other (unmasked) tokens via a parallel multi-head attention mechanism, allowing the signal for key tokens to be amplified and less important tokens to be diminished.



Recap: Transformer Architecture

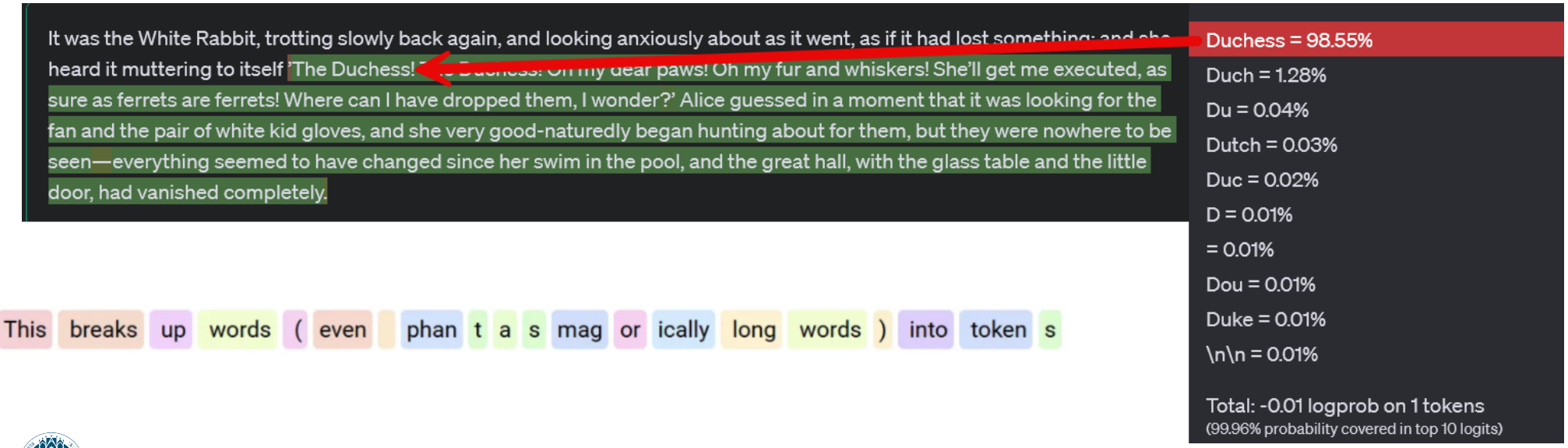
- A type of neural network that:
  - understands **relationships between words** in a **sequence all at once** (not one by one)
  - looks at all words **in relation to each other** and focus on the parts that matter most using the **attention heads**

The **animal** didn't cross the **street** because **it** was too tired.



Recap: What Do LLMs Actually Do?

LLMs do **next token prediction**. They predict the next token in a sequence of tokens (**context**) based on their **training data**. Each predicted token becomes part of the context for the next prediction (**autoregressive**). This simple mechanism, scaled up massively, produces the behaviors we observe.



Part I: Natural Language Processing

Why is NLP Important?

- Goal: enable computers to **understand**, **interpret**, and **generate** human language in a way that is both **meaningful** and **useful**
- Communication with a machine using **natural** instead of **artificial** language
- Automating tasks
- Applications: speech recognition, spam detection, sentiment analysis, machine translation

Issues with Modeling Natural Language

- Ambiguity
- Bias
- Cultural and regional differences
- Accents, dialects, individual differences

NLP Techniques and Tasks

- **Techniques**:
  - Text preprocessing: tokenization, stemming, lemmatization
  - Text representation: **word embeddings**, **TF**, **IDF**, **TF-IDF**
  - Language modeling: N-grams, **Transformer models**

- **Tasks**:
  - POS tagging
  - Named entity recognition
  - Coreference resolution
  - Sentiment analysis
  - **Keyword extraction**
  - **Text generation**
  - Topic modeling
  - **Machine translation**
  - Text classification
  - Text summarization
  - Question answering

NLP Pipeline: Preprocessing, Representation, and Modeling

- **Text preprocessing**: cleaning and preparing text
  - Examples: tokenization, lowercasing, stop-word removal, stemming, lemmatization
- **Text representation**: converting text into numbers
  - Examples: BoW, IDF, TF-IDF, word embeddings
- **Language modeling**: modeling the probability of word sequences
  - Examples: n-gram language models, neural language models, transformers

Text Mining and Text Preprocessing

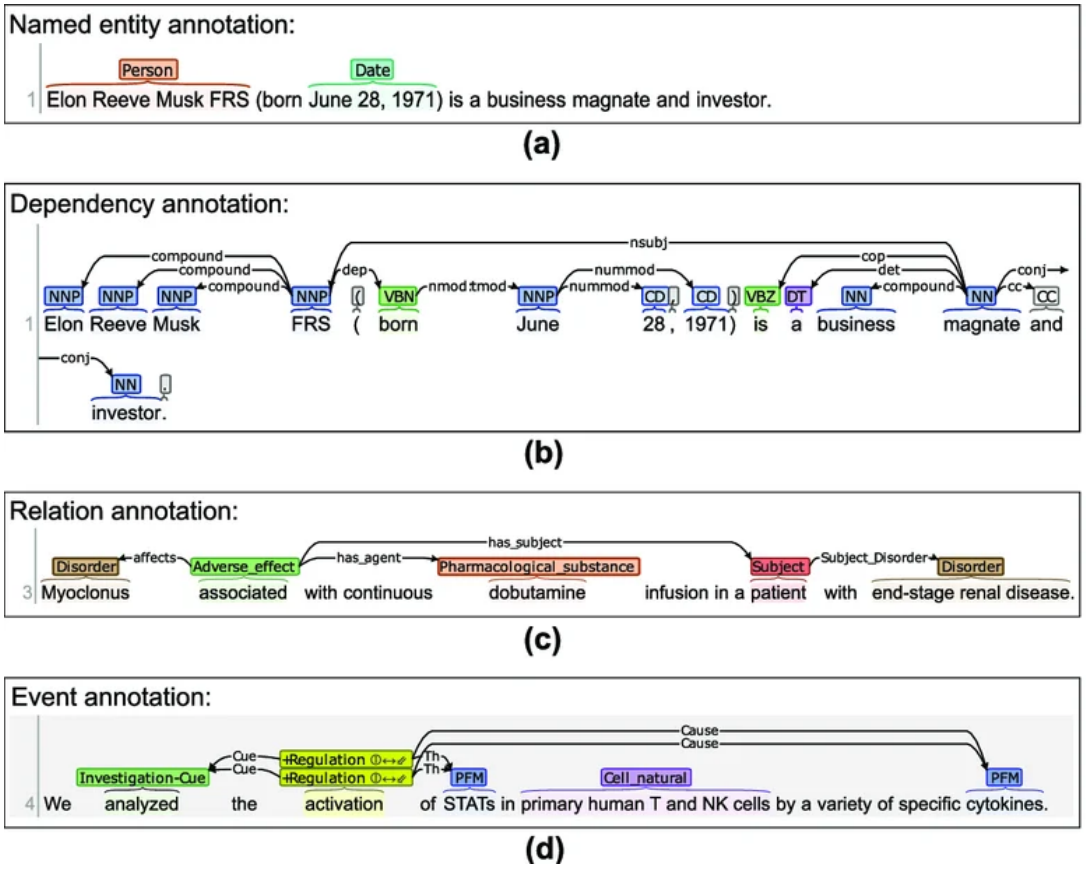
- Issue: language is **noisy**, **inconsistent**, and **unstructured**
- Goal: **cleaning** and **preparing** the raw text data for analysis

Text Mining: Corpus

- Definition: **collections** of texts (documents), first level of introducing **structure**
- Type of corpus: text, images, video, audio
- Type of data: books, documents, children's texts, movie subtitles, tweets, reddit repo etc.
- Usually they come with **meta-information** (forum threads, topics etc.)
- Usually in the same language

Text Mining: Corpus Annotation

- The “units” or fractions of “units” of text have been annotated (marked) with additional information in order to be used as a **training set** for a specific applications
- Lots of text mining relies on humans annotating the corpora



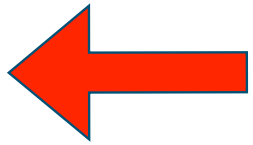
Abdullah, M.H.A., Aziz, N., Abdulkadir, S.J., Alhussian, H.S.A. and Talpur, N., 2023. Systematic literature review of information extraction from textual data: recent methods, applications, trends, and challenges.

Text Mining: Text Preprocessing

- Once the corpus is obtained (annotated or not), it has to go through the **preprocessing** stage
- Going from **unstructured** to **structured** data (from text to, ideally, numbers)
- Tokenization, morphological normalization, stemming, lemmatization, stop word removal, BOW, IDF, TF-IDF etc.

NLP Pipeline: Preprocessing, Representation, and Modeling

- **Text preprocessing**: cleaning and preparing text
  - Examples: tokenization, lowercasing, stop-word removal, stemming, lemmatization



- **Text representation**: converting text into numbers
  - Examples: BoW, IDF, TF-IDF, word embeddings
- **Language modeling**: modeling the probability of word sequences
  - Examples: n-gram language models, neural language models, transformers

Text Preprocessing: Tokenization and Tokenizers

- Tokenization transforms text into **numerical units** for LLM processing. The tokenization strategy **prioritizes computational efficiency** by minimizing sequence length

A token is the atomic unit for LLMs
(100 tokens ≈ 75 English words)

Raw internet text: “Hello World!”

Cleaning and filtering (removes spam, deduplication)

Tokenizer: [‘Hello’, ‘World’, ‘!’]

Token IDs: [13225, 5922, 0]

Text Preprocessing: Tokenization and Tokenizers

- Splitting text into **smaller units** (sentences, words, or word forms)
- Tokenization is **application dependent**: not just splitting text into words, specifically designed for certain **task** and **language**
- Differences in tokenizers: BOS/EOS tokens, multilingual
- What about words like “Sehenswürdigkeiten”?

They’ve been celebrating Casper van der Veldren’s 65th birthday in Los Angeles.

Text Preprocessing: Tokenization and Tokenizers

Joiner	Token sequence (conceptual)	Token count
" —"	[" —"]	1
", and "	[" ,", " and"]	2

Let's talk about em dashes in AI. Maria Sukhavera. https://msukhareva.substack.com/p/lets-talk-about-em-dashes-in-ai

Text Preprocessing: Tokenization and Tokenizers

- Check out the tokenizers, try out different examples in different languages:

https://platform.openai.com/tokenizer

https://huggingface.co/spaces/Xenova/the-tokenizer-playground

https://gpt-tokenizer.dev/

Text Preprocessing: Stop Word Removal

- **Function** vs **content** words
- Removing words that are not informative (and, but, of, is, are)
- Many NLP packages (e.g. NLTK, SpaCy in Python) have predetermined stop word lists for specific languages BUT there is **no one preset rule**
- Stop words lists can be **aggressive** or **conservative**
- Specifically important with **word frequencies**

Text Mining: Stop Word Removal

- Why is removing stop words important?
- Sentences can be different in terms of meaning regardless of the overlapping words

The cats, which were seven, started to climb the tree.

The Saxons, which were outnumbered, started to prepare the siege.

Text Preprocessing: Lemmatization vs Stemming

- **Lemmatization**: output is an existing, normalized form of a word that can be found in a dictionary (lemma)
- **Stemming**: cuts the end form of a token (cutting out the suffixes)

Both are there to reduce inflectional forms of a word to a common base form

Text Preprocessing: Lemmatization vs Stemming

“caring”: lemmatization “care"; stemming “car”

“stripes”: lemmatization “strip" (in a verb context), “stripe" (in a noun context); stemming “strip"

“walking”, “running” “swimming”: in both cases “walk”, “run”, “swim”

Text Preprocessing: Lemmatization vs Stemming

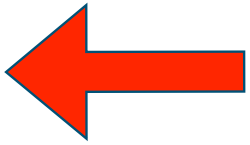
- Which one is better? It depends…
- Lemmatization is more **computationally expensive** (involves look-up tables) but more **accurate**
- Stemming is **less accurate** but **easier to compute**
- With **large datasets** it is better to go with **stemming** (accuracy vs cost)

Text Preprocessing: Other Forms of Normalization

- Case-folding: lowercase, uppercase
- Alternative spelling: AmE vs BrE
- Transliterations: ö - oe, ä - ae, ü - ue

NLP Pipeline: Preprocessing, Representation, and Modeling

- **Text preprocessing**: cleaning and preparing text
  - Examples: tokenization, lowercasing, stop-word removal, stemming, lemmatization
- **Text representation**: converting text into numbers
  - Examples: BoW, IDF, TF-IDF, word embeddings



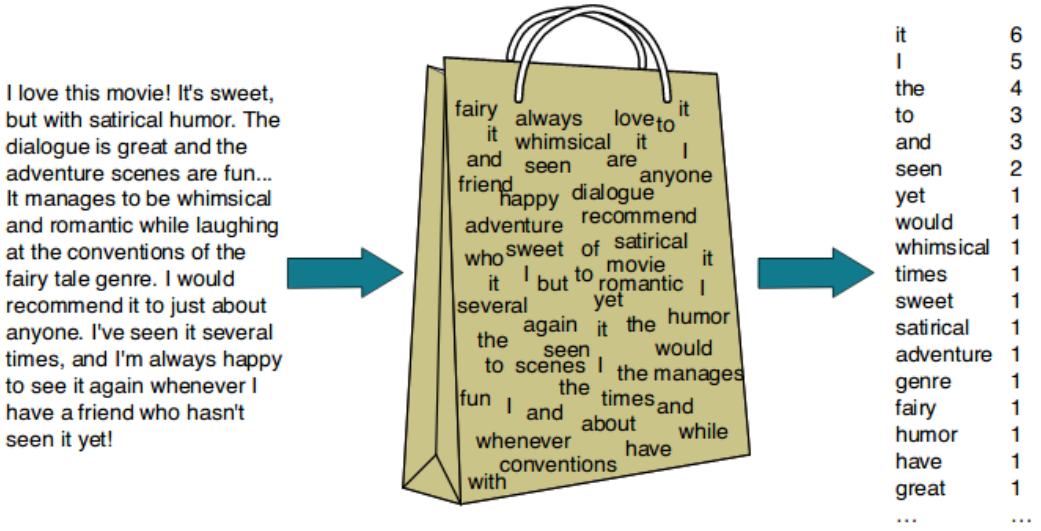
- **Language modeling**: modeling the probability of word sequences
  - Examples: n-gram language models, neural language models, transformers

Text Representation: Frequency-based Methods

- BOW: Bag of Words
- IDF: Inverse Document Frequency
- TF-IDF: Term Frequency - Inverse Document Frequency
- Essential in **information retrieval**
- BoW, IDF, and TF-IDF are **vectorization** or **feature extraction** methods that transform text into **numerical representations**

Text Representation: Bag of Words

- Represents documents in corpora as bags of tokens
- After removing stop words and applying other forms of token normalization we get a **word vector**
- A method of indicating **token frequencies**
- Does not consider the **word order**



Text Representation: Bag of Words

Bag of Words (BoW)

Let:

- $D = \{d_1, d_2, \ldots, d_N\}$ be a collection of documents
- $V = \{t_1, t_2, \ldots, t_M\}$ be the vocabulary

Then the **Bag of Words** representation of a document d is:

$$\mathbf{x}^{(d)} = [c(t_1, d), c(t_2, d), \ldots, c(t_M, d)]$$

where:

- $c(t_i, d)$ = count of term t_i in document d

Text preprocessing is important for text analysis

Vocabulary tokens:
["text", "preprocessing", "is", "important", "for", "text", "analysis"]

Stop-word removal

```
{
  "text": 2,
  "preprocessing": 1,
 "important": 1,
 "analysis": 1
}
```

BoW: [2,1,1,1]

Text Representation: Bag of Words

- Many sentences have **the same BoW representation**

James loves watching movies, **Katie** hates it.

Katie loves watching movies, **James** hates it.

- Easy to implement
- Intermediate step before applying more **complex methods**
- Works **well enough** in many applications (IR, spam detection)

Text Representation: Inverse Document Frequency

- How **specific** a word is in a corpus
- IDF is always 0 if the word occurs in every document
- Words with **low IDF score** appear in many documents and are **not useful** in distinguishing between them
- BoW and TF-IDF scoring provide **instances** (typically documents) with words as features, but they **ignore the word order**

Text Representation: Inverse Document Frequency

Inverse Document Frequency (IDF)

Let:

- N = total number of documents
- $\mathrm{df}(t)$ = number of documents containing term t

Then:

$$\mathrm{idf}(t) = \log\left(\frac{N}{\mathrm{df}(t)}\right)$$

A smoothed version often used is:

$$\mathrm{idf}(t) = \log\left(\frac{N+1}{\mathrm{df}(t)+1}\right) + 1$$

The **higher** the IDF score, the more **rare** the word is in a corpus

Text Representation: Inverse Document Frequency

Example 1

For the word **"Merkel"**:

$$\text{idf}(\text{"Merkel"}) = \log_2\left(\frac{100}{100}\right) = 0$$

This means the word appears in all 100 articles, so it carries little distinguishing information.

Example 2

For the word **"Seehofer"**:

$$\text{idf}(\text{"Seehofer"}) = \log_2\left(\frac{100}{10}\right) \approx 3.32$$

This means the word appears in only 10 out of 100 articles, so it is more informative.

Example 3

For the word **"Schulz"**:

$$\text{idf}(\text{"Schulz"}) = \log_2\left(\frac{100}{1}\right) \approx 6.64$$

This means the word appears in only 1 out of 100 articles, so it is highly informative.

Text Representation: TF-IDF: Term Frequency - Inverse Document Frequency

Term Frequency–Inverse Document Frequency (TF-IDF)

Let:

- $\mathrm{tf}(t, d)$ = term frequency of term t in document d
- $\mathrm{idf}(t)$ = inverse document frequency of term t

Then:

$$\mathrm{tfidf}(t, d) = \mathrm{tf}(t, d) \cdot \mathrm{idf}(t)$$

If raw count is used for term frequency:

$$\mathrm{tf}(t, d) = c(t, d)$$

so:

$$\mathrm{tfidf}(t, d) = c(t, d) \cdot \log\left(\frac{N}{\mathrm{df}(t)}\right)$$

The **higher** the TF-IDF score, the **more important** a word is in a corpus

Text Representation: TF-IDF: Term Frequency - Inverse Document Frequency

Document 1:
Cats are the only pets in the feline family, while dogs are candids.

Document 2:
Cats are the third most popular pet in the US.

Document 3:
Dogs have been selected for millennia as pet animals.

Document 4:
Normally, dogs are not aggressive towards other dogs outside their territory.

Feline

tf = 1 (in Document 1 the word occurs one time)

idf = log(4/1) = 2 (4 documents, 1 doc contains the word “feline”)

tfidf = 1*2 = 2

Text Representation: TF-IDF: Term Frequency - Inverse Document Frequency

Document 1:
Cats are the only pets in the feline family, while dogs are candids.

Document 2:
Cats are the third most popular pet in the US.

Document 3:
Dogs have been selected for millennia as pet animals.

Document 4:
Normally, dogs are not aggressive towards other dogs outside their territory.

Cat

tf = 1 (in Document 2 the word occurs one time)

idf = log(4/2) = 1 (4 documents, 2 docs contain the word “cat”)

tfidf = 1*1 = 1

Text Representation: TF-IDF: Term Frequency - Inverse Document Frequency

Document 1:
Cats are the only pets in the feline family, while dogs are candids.

Document 2:
Cats are the third most popular pet in the US.

Document 3:
Dogs have been selected for millennia as pet animals.

Document 4:
Normally, dogs are not aggressive towards other dogs outside their territory.

Dog

tf = 2 (in Document 4 the word occurs one time)

idf = log(4/3) = 0.41 (4 documents, 3 docs contain the word “dog”)

tfidf = 2*0.41 = 0.82

Text Representation Task: BoW, IDF, TF-IDF

Document 1:
Process mining improves process analysis Process

Document 2:
Process discovery improves business analysis

Exercise 1: Write BoW for the first document
Exercise 2: For the word “process” in Document 1 calculate TF, IDF, TF-IDF

Text Representation Task: BoW, IDF, TF-IDF

Document 1:
process mining improves process analysis

Document 2:
process discovery improves business analysis

[“process”, “mining”, “improves”, “process”“analysis”]

BoW D1: [2, 1, 1, 1]

Process

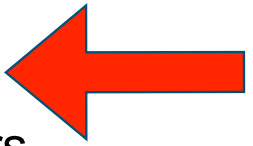
tf = 2

idf = log(2/2) = 0 (2 documents, 2 docs contain the word “process”)

tfidf = 2*0 = 0

NLP Pipeline: Preprocessing, Representation, and Modeling

- **Text preprocessing**: cleaning and preparing text
  - Examples: tokenization, lowercasing, stop-word removal, stemming, lemmatization
- **Text representation**: converting text into numbers
  - Examples: BoW, IDF, TF-IDF, word embeddings
- **Language modeling**: modeling the probability of word sequences
  - Examples: n-gram language models, neural language models, transformers



Language Modeling: n-grams

Sequences of items (words, characters, or tokens) extracted from a given text **sample**

Apples are good for you

- **Unigram**: ‘apples’, ‘are’, ‘good’, ‘for’, ‘you’ (same as BoW)
- **Bigram**: [('apples', 'are'), ('are', 'good'), ('good', 'for'), ('for', 'you')]
- **Trigram**: [('apples', 'are', 'good'), ('are', 'good', 'for'), ('good', 'for', 'you')]

Language Modeling: n-grams

- **Unigram**: P(phone)
- **Bigram**: P(phone|cell)
- **Trigram**: P(phone|your cell)

Next word prediction is based on conditional probabilities

P(X|Y)

probability of next word based on the text that is already there

Part II: (Large) Language Models

Language Models After Transformers

LLMs as NLP Tools
Machine Learning

Sentiment Analysis
“XSLT is terrible” → Negative

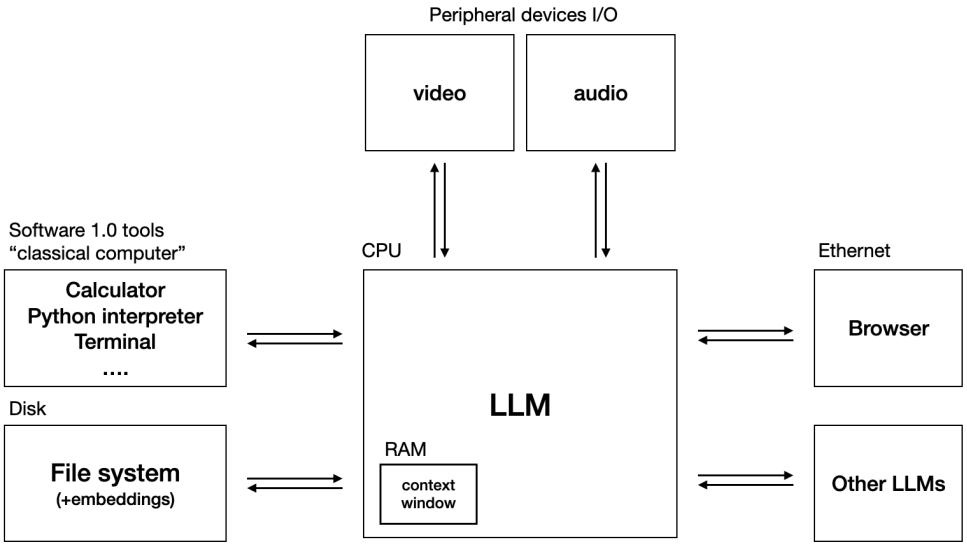
Named Entity Recognition
“Apple released the iPhone in Cupertino” →
[Apple: ORG] [iPhone: PRODUCT] [Cupertino: LOC]

Translation
"Bonjour" → "Hello"

Summarization
Long article → Short summary

Classification
Email → Spam/Not Spam

(Frontier-) LLM as Operating System
Applied Generative AI



“Analyze this Excel file, create visualizations, and write a report” → LLM reads file, runs Python for analysis, generates charts, writes document

What Is a Language Model?

- Definition 1:

A statistical model for the probability of a word sequence W

$$P\ W = P(w1\ ,\ \ldots\ ,\ wn)$$

- Definition 2:

A statistical model for the probability of a word given a context, with context being for example

- preceding words $P\ wn{+}1\ w1\ ,\ \ldots\ ,\ wn)$
- surrounding words $P\ wi\ w1\ ,\ \ldots\ wi{-}1,\ wi{+}1,\ \ldots\ ,\ wn)$

What Is a Language Model?

Language Modeling

- LM: probability distribution over sequences of tokens/words $p(x_1, \ldots, x_L)$

P(the, mouse, ate, the, cheese) = 0.02

P(the, the, mouse, ate, cheese) = 0.0001 Syntactic knowledge

P(the, cheese, ate, the, mouse) = 0.001 Semantic knowledge

Chatbots are autoregressive language models: each predicted token becomes part of the context for the next prediction

Language Models Before Transformers

- **Next word prediction** based on **word frequencies**
- Bigram predictions e.g., P(phone|cell)
- Shallow neural networks
- Nonsense output: older language models could often produce words that fit locally, but they struggled to maintain meaning, world knowledge, and long-range coherence.

Language Models Before Transformers

- Older language models often produce words that fit locally, but they struggle to maintain meaning, world knowledge, and long-range coherence

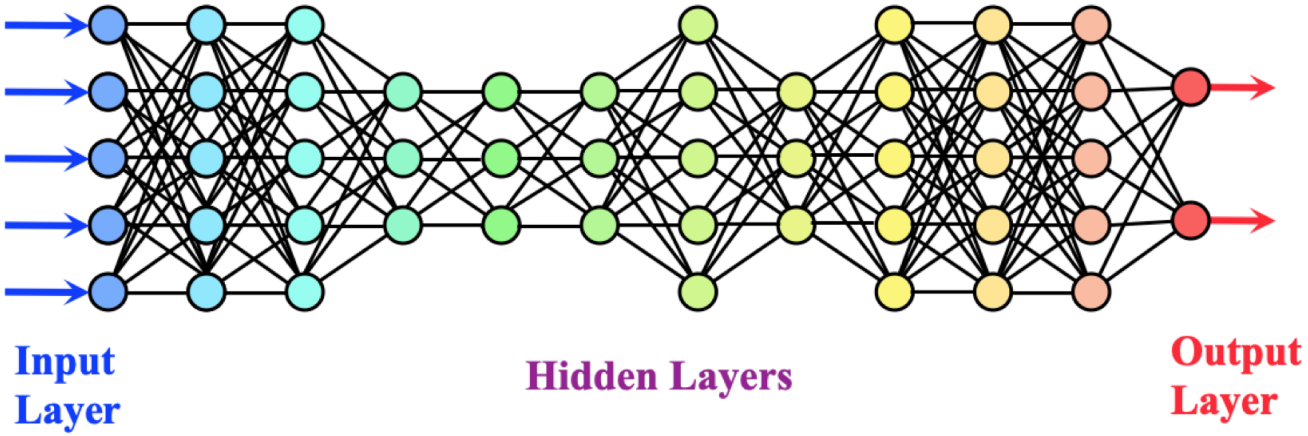
“cell phone” is a common bigram
“was very” is common
“answered the question” is common

The cell phone was very delicious because the weather answered the question

Language Models After Transformers

- Deep Neural Networks

We can have more than one hidden layer

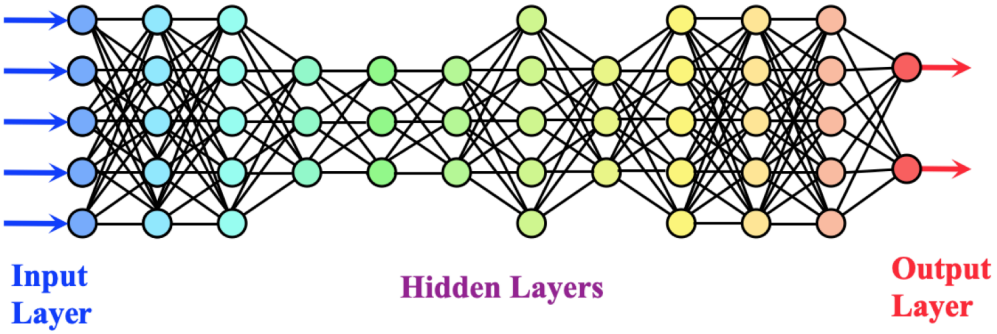


... and add all kinds of extra components

Large Language Models: Parameters

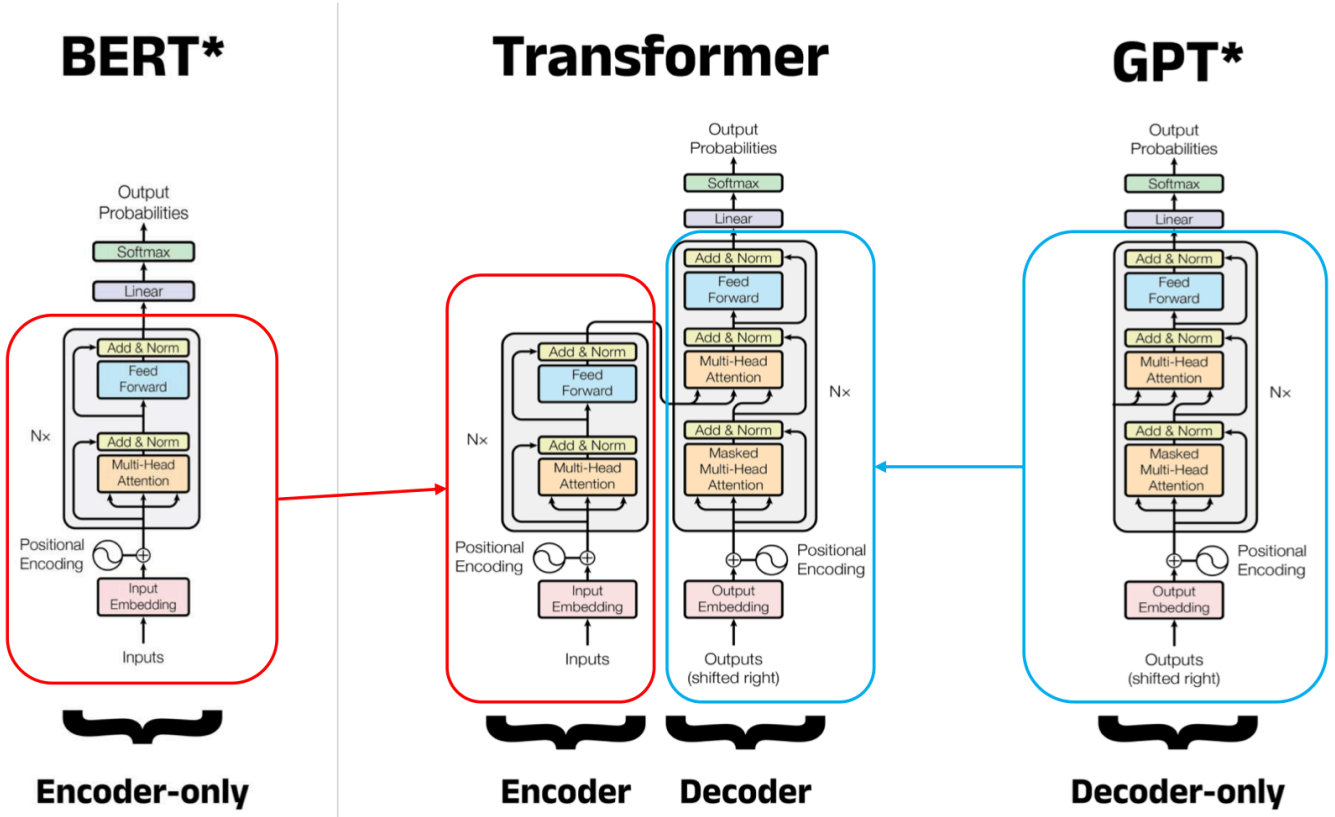
- Each layer of neural network is responsible for something else
- A mathematical weight that determines the strength of a connection

We can have more than one hidden layer



... and add all kinds of extra components

Types of LLMs



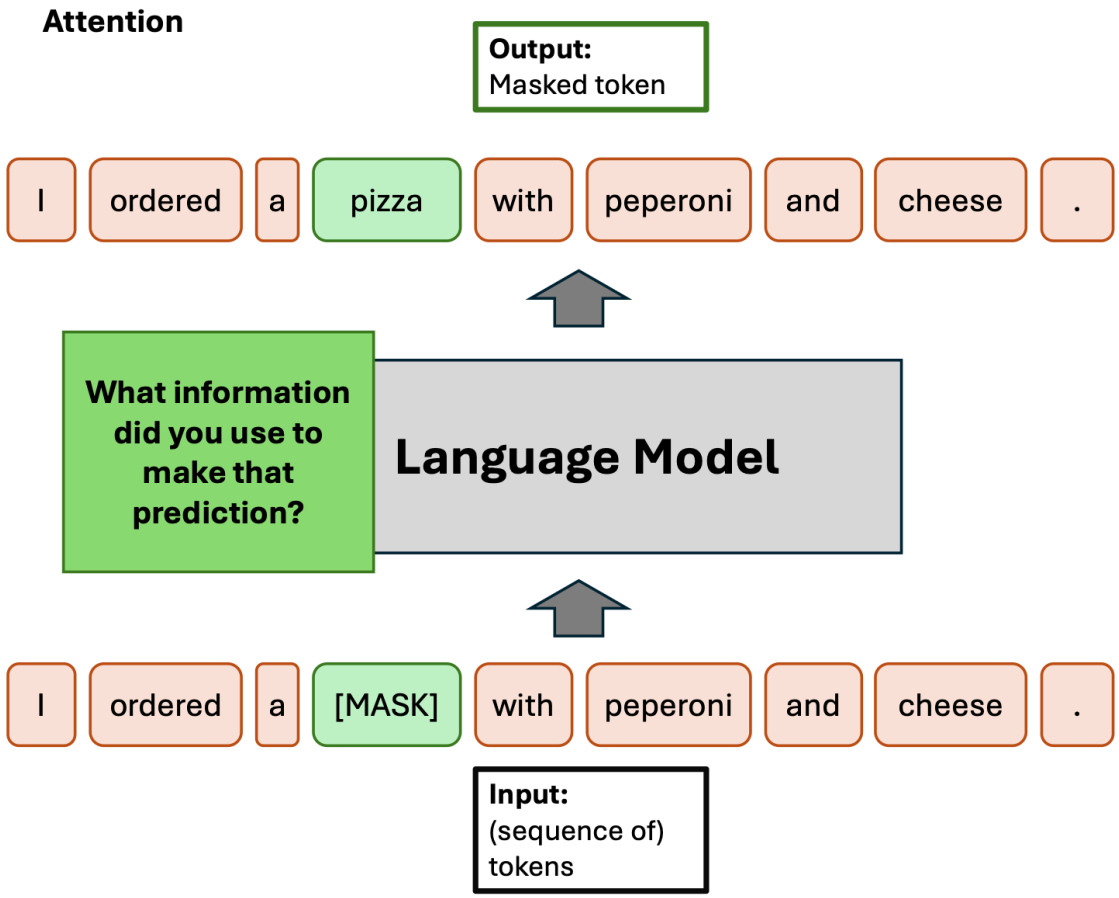
Predict the **correct** word

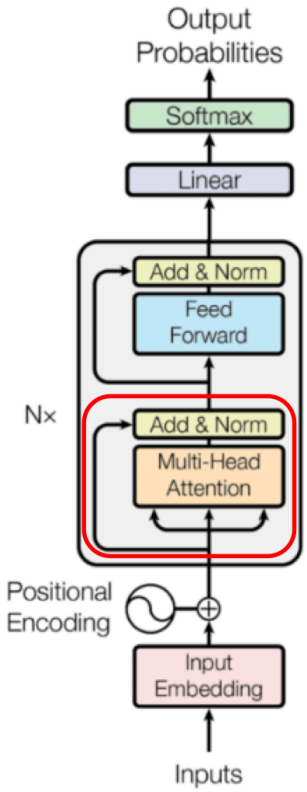
Predict the **next** word

Types of LLMs

- **GPT**: “Given the words so far, what comes next?” → **autoregressive**
- **BERT**: “Given the full sentence with a missing word, what fits here?” → **not autoregressive**

Types of LLMs: Encoder-only, BERT





Types of LLMs: Encoder-only, BERT

- Masked models are better for:

Named entity recognition

“Marie Curie was born in Warsaw.”

Detect: `Marie Curie = person`, `Warsaw = location`

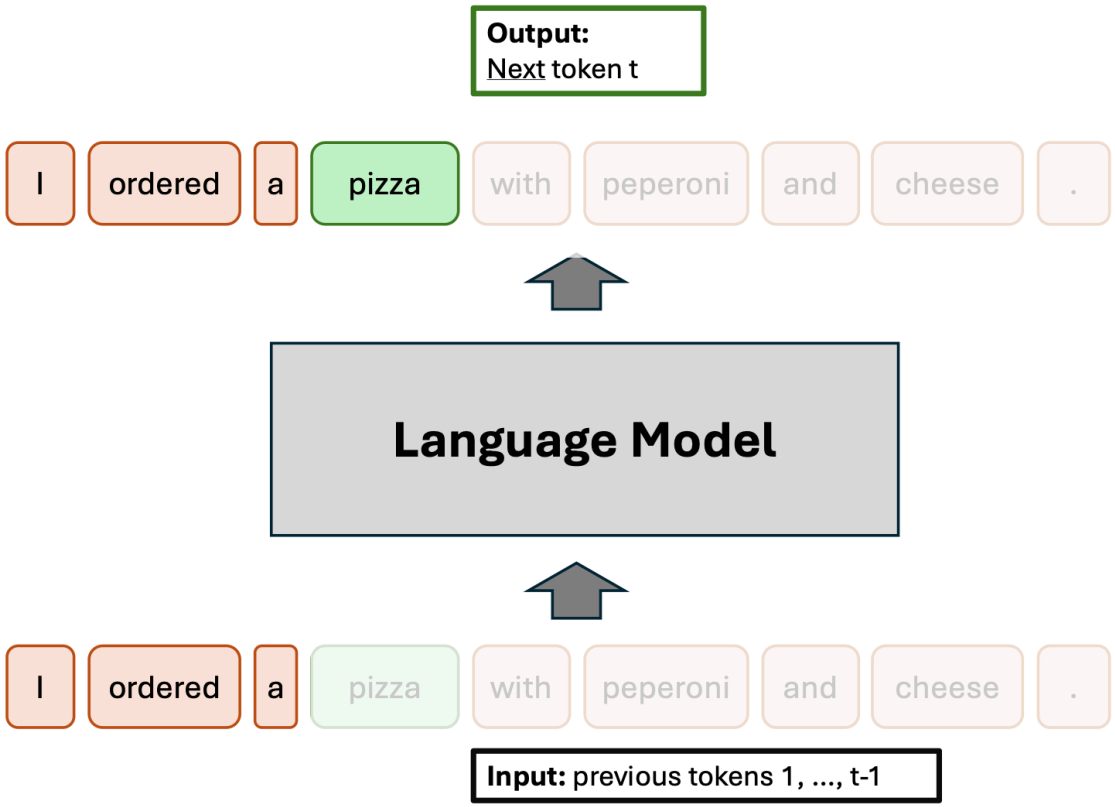
Text classification

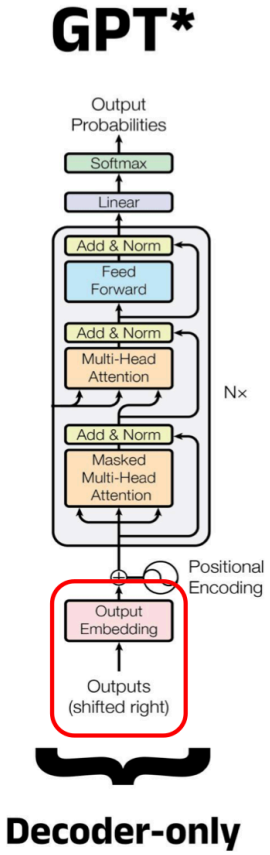
Is this review positive or negative? Is this email spam or not?

Extractive question answering

“Where was Marie Curie born?”
Text: “Marie Curie was born in Warsaw.”, Answer: “Warsaw”

Types of LLMs: Decoder-only, GPT





Types of LLMs: Decoder-only, GPT

- Decoder-only models are better for:

Text generation

Write an explanation of transformers

Dialogue and chatbots

Answer the user's question in natural language

Summarization

Turn this long article into a short summary

Translation

Translate this paragraph into German

Code generation

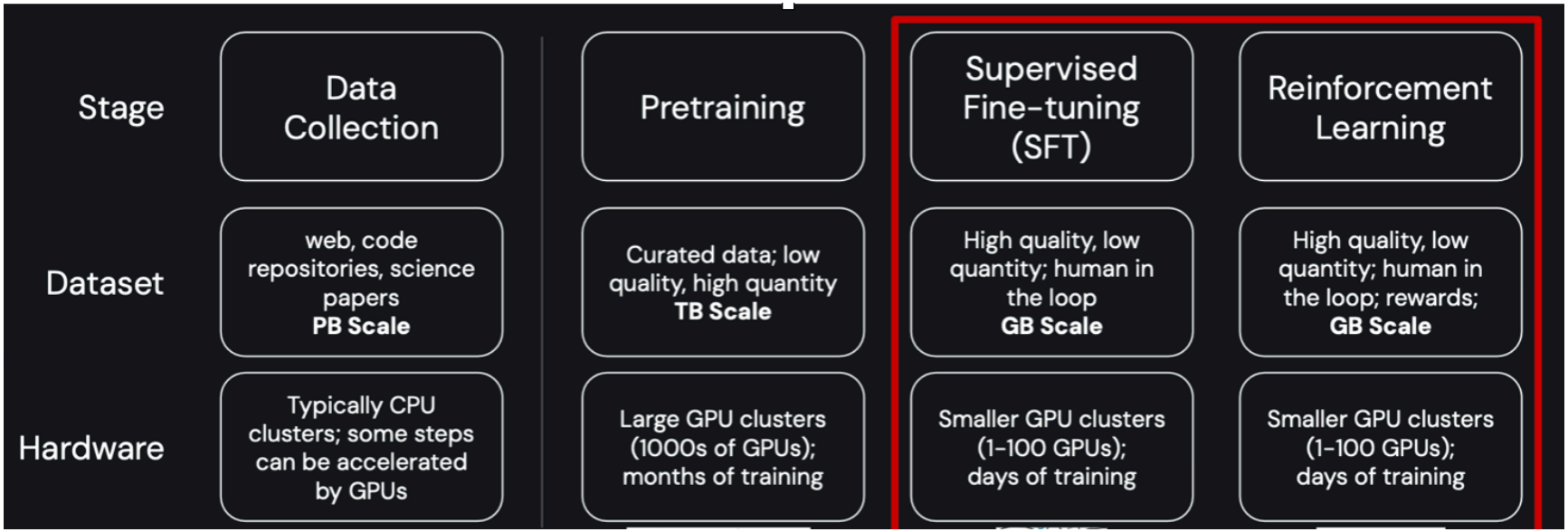
Write Python code for a function

LLM Training

- Can be broken down into three stages:

    1. Data **preparation** and **preprocessing**
    2. Base model **pretraining**
    3. **Fine-tuning** with alignment

LLM Training



LLM Training: Data Preparation and Preprocessing

- Begins with **selecting appropriate training corpora** (LLMs require extremely large volumes of text)
- Common sources: books, code repositories, academic papers, large-scale web crawls (Common Crawl, Wikipedia)
- Once collected, the data undergoes several **preprocessing steps**

LLM Training: Data Preparation and Preprocessing

- **Preprocessing steps** before the **model training**:
  - **Quality filtering**: removing low-quality, biased, offensive, or harmful content
  - **Deduplication**: eliminating repeated text
  - **Privacy scrubbing**: anonymizing text and removing sensitive or confidential information
  - **Tokenization**: converting text into tokens suitable for the model
  - **Formatting and segmentation**: splitting text into fixed-length sequences (e.g., 1k–4k tokens) for training

LLM Training: Base Model Pretraining

- Choosing model architecture (commonly the transformer architecture)
- The central objective is (masked) **language modeling**: the model develops rich representations of language from the training data (grammar, semantics, sentence structure, and vocabulary)
- In transformers, **positional embeddings** (PE) help the model determine the correct placement of each word

LLM Training: Base Model Pretraining

- Language modeling itself is based on **predicting the next token** from the surrounding **context**
- The effectiveness of pretraining depends heavily on **scaling**, which requires balancing model size, data volume, and available compute resources

LLM Training: Post-training

- The chatbots we use today are no longer pure language models
- **System prompts** (each session starts with a standardized prompt that ensures certain „behavior“/“personality“)
- **Chain-of-thought** (the reasoning steps, „Thinking…“)
- **Multilingual**
- **Multimodal** (trained on text + images)
- Many proprietary models are **not open** (i.e., model internals like weights, embeddings, probabilities etc. are not freely available)

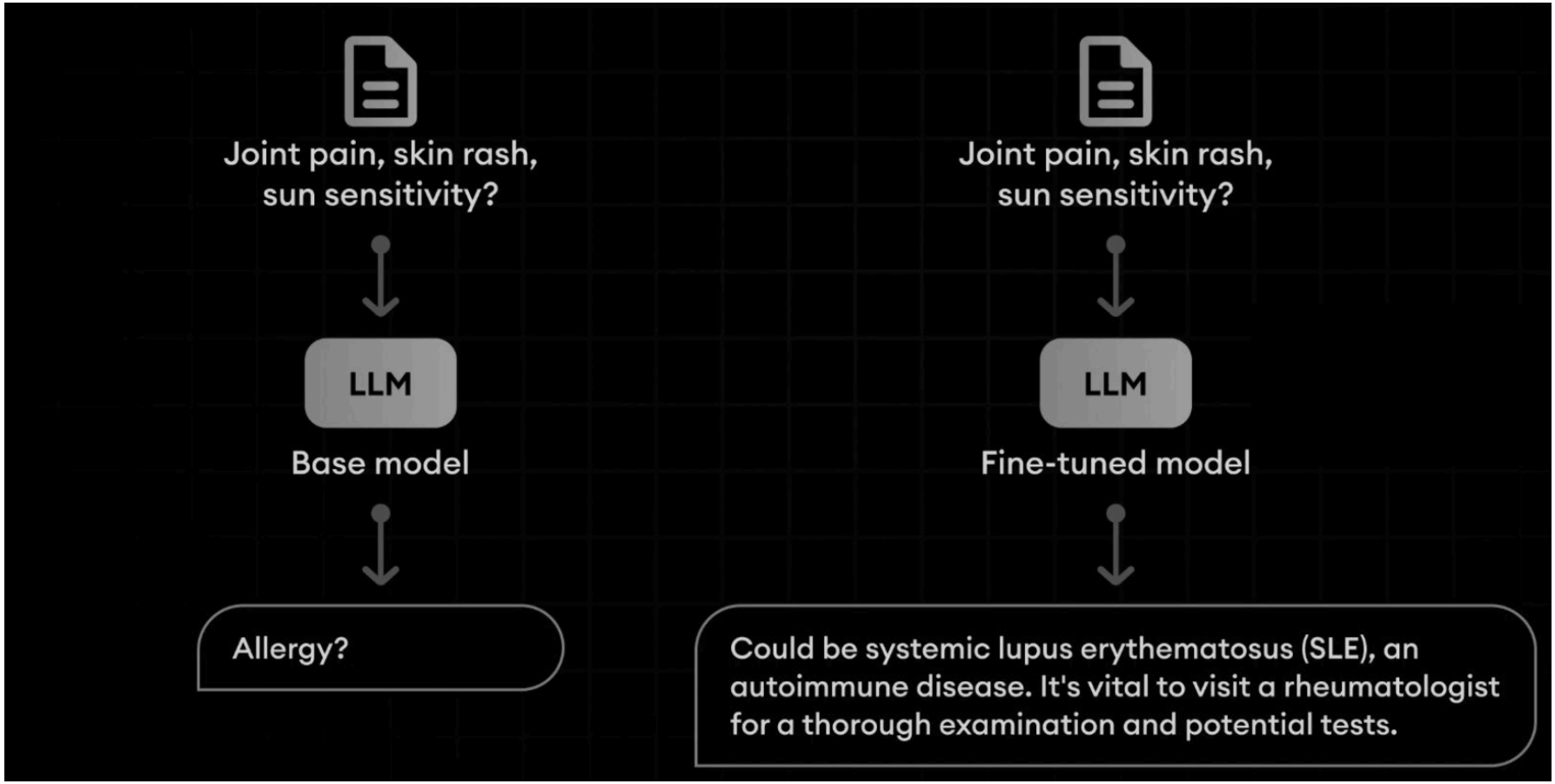
LLM Training: Fine-Tuning and RLFH

- At the end of pretraining, the resulting model is **powerful** but **general**: it has absorbed vast knowledge from data but is not **specialized for tasks**, may generate **irrelevant** or **unsafe** text, and often **lacks alignment** with human expectations.
- To address this, the model undergoes **fine-tuning** often using **RLFH**, which can take several forms
- Fine-tuning: **adjusting model's weights** on specific corpus

LLM Training: Fine-Tuning and RLFH

- **Fine-tuning**: adapting the pretrained model to specific tasks or domains using labeled datasets e.g., for question answering or for specialized fields
- **Alignment tuning**: ensuring the model's responses are helpful (useful for tasks), honest (factually accurate), and harmless (safe for users and society) using RLHF, where a reward model is trained based on human preference rankings
- **Parameter-efficient tuning**: reducing compute and memory costs by updating only a small subset of parameters while keeping most of the pretrained model frozen e.g. Low-Rank Adaptation (LoRA) (performs comparably to full fine-tuning)
- **Safety fine-tuning**: improving safety by combining RLHF with supervised fine-tuning. The model is exposed to high-risk prompts, unsafe responses are flagged, and its ability to handle safety-critical scenarios is strengthened

LLM Training: Fine-Tuning with Alignment



LLM Context Window

Model Context Window = 8K

Input Token 6000 Token

A context window, in the context of large language models (LLMs), refers to the portion of text that the model can consider at once when generating or analyzing language.
[...]

Output Token 1500 Token

Lorem ipsum …

Context Window = 6000 + 1500 < 8000

Model Context Window = 8K

Input Token 10000 Token

A context window, in the context of large language models (LLMs), refers to the portion of text that the model can consider at once when generating or analyzing language. It is essentially the window through which the model "sees" and processes text, helping it understand the current context to make predictions, generate coherent sentences, or provide relevant responses.
[...]

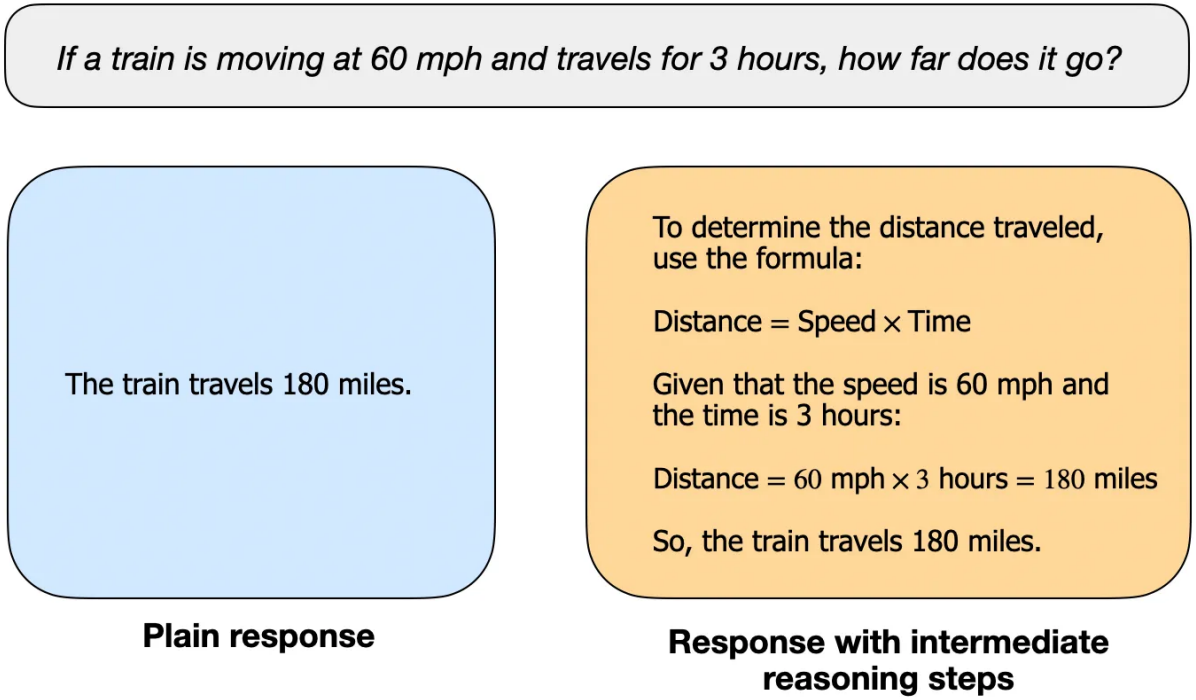
Output Token 1500 Token

Lorem ipsum …

Context Window = 10000 + 1500 > 8000
3500 tokens are not in the context window!

Large Language Models: Reasoning Models

- Inference-time scaling: increasing **computational resources** during inference to improve output quality
- In **proprietary models** specific methods are not disclosed



Practical Session: Text Preprocessing Steps in Practice

- Go to Github
- Open “session2_NLP_techniques.ipynb”
- Click on “open in google collab”

Wrap-Up & Q&A

- NLP pipeline includes **text preprocessing**, **text representation**, and **language modeling**
- In NLP, there is **no universal best solution**: preprocessing steps, model architecture, and training choices must be adapted to the **specific task** and **dataset**
- Modern LLMs are no longer just **classical language models**, but complex systems shaped by pretraining, deep architectures, fine-tuning, instruction tuning, and RLHF
- Question from me: Would you like to continue with NLP if we have time?

Next Up

- Presentation “Durably reducing conspiracy beliefs through dialogues with AI” by Isabella Barth López chaired by Fanny Burkhardt
- Room change
- How Large Language Models Represent Meaning
  - Distributional Semantics
  - Word Embedding Models
  - Semantic Spaces
  - Practical Part

Thank you!

UNIVERSITY
OF COLOGNE

https://selflearningsystems.uni-koeln.de/

@hannawoloszyn.bsky.social